



ANALYTICS ENGINEERING PROJECT

End-to-End YouTube Analytics Pipeline

A modern ELT pipeline extracting, loading, transforming and visualizing YouTube data.

3 channels 719K subscribers 42.9M views 513 videos

Python

YouTube Data API v3

BigQuery

dbt

Looker Studio

EXTRACT → LOAD → TRANSFORM → VISUALIZE

PROJECT OVERVIEW

Project Overview

OBJECTIVE

Build an end-to-end YouTube Analytics platform that automates data ingestion, transformation, and reporting using modern analytics engineering tools.

Data ingestion

Dimensional modeling

Analytics engineering

Business intelligence



Extract

Pull channels, videos, comments & playlists from the YouTube Data API v3.



Load

Land raw JSON into BigQuery staging and persistent raw tables.



Transform

Model with dbt into clean dimensions, facts and reporting marts.

TOOLING

Tech Stack



Python

Extraction & loading scripts



YouTube Data API v3

Source of all raw data



Google Cloud

Auth, keys & platform



BigQuery

Cloud data warehouse



dbt

Layered SQL transformations



Looker Studio

Interactive dashboards



Git & GitHub

Version control

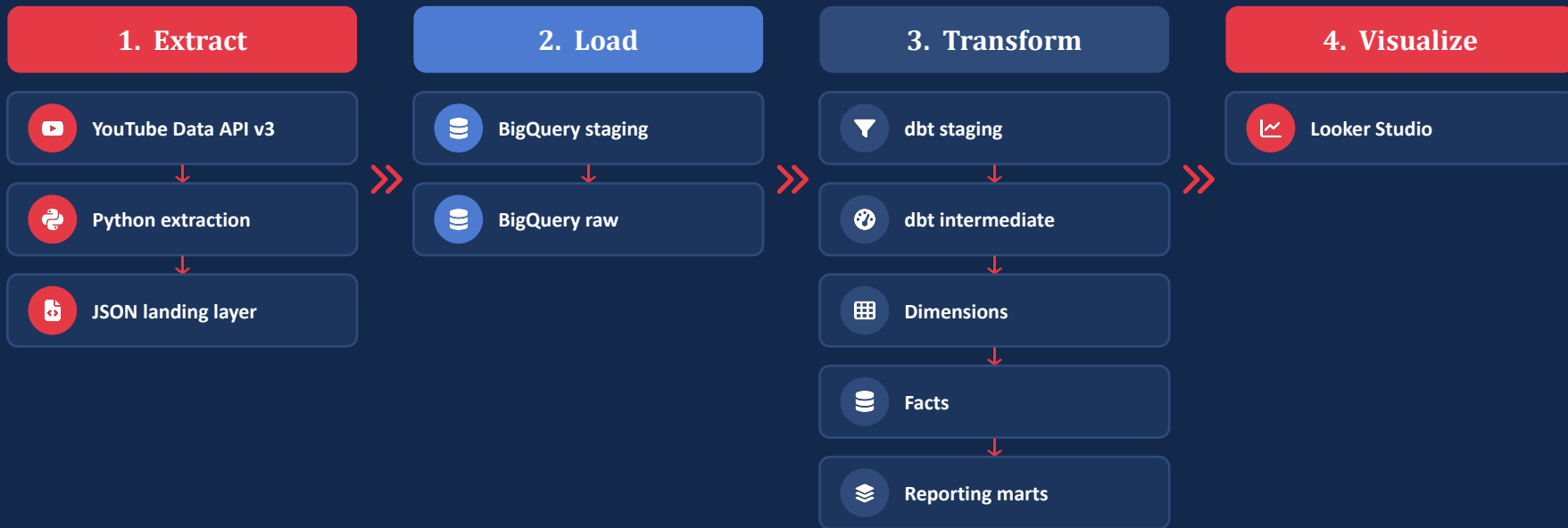


Apache Airflow

Orchestration

NEXT PHASE

End-to-End Architecture



A single flow from raw API responses to business-ready dashboards.

PHASE 1

Data Extraction



Authentication

GCP project + YouTube Data API v3, authenticated with a generated API key.



Channel selection

Three educational channels: Ankit Bansal, Ansh Lamba and techTFQ.



resolve_channels.py

Handles → Channel IDs + Uploads Playlist IDs, stored in a YAML config so new channels need no code changes.

ENTITIES EXTRACTED



Channels

Subs · views · video count



Videos

Title · duration · views · likes



Comments

Author · text · likes · replies



Playlists

Details & playlist-video mapping



Categories

Category ID → category name

Why Store JSON First?



LANDING STRUCTURE

`data/raw/`

- `| channels/`
- `| videos/`
- `| comments/`
- `| playlists/`
- `| categories/`

Each API response is written to disk as a JSON snapshot before loading.

Storing raw JSON as a landing layer gives the pipeline resilience and repeatability:



Decouples extraction from loading



Reruns without re-calling the API



Reduces YouTube API quota usage



Simplifies debugging & backups



Enables reprocessing when logic changes

In production this landing layer is replaced by Google Cloud Storage or Amazon S3.

PHASE 2

Loading into BigQuery



Read & parse the JSON `reader.py`

Loads JSON snapshots from disk and returns clean Python objects.



Orchestrate the load `loader.py`

Transforms records, ensures tables exist, loads staging, then merges into raw — reused for every entity.



Handle all BigQuery I/O `bigquery.py`

Dataset & table creation, schemas, load jobs and the merge logic.

TWO-TABLE PATTERN

`videos_stage`

Transient · holds only the current batch · overwritten every run



`videos_raw`

Persistent history · source for dbt · one row per business key + snapshot

Merge Strategy

BigQuery's free tier doesn't support MERGE — so the pipeline simulates it. Here's the same video across two runs:



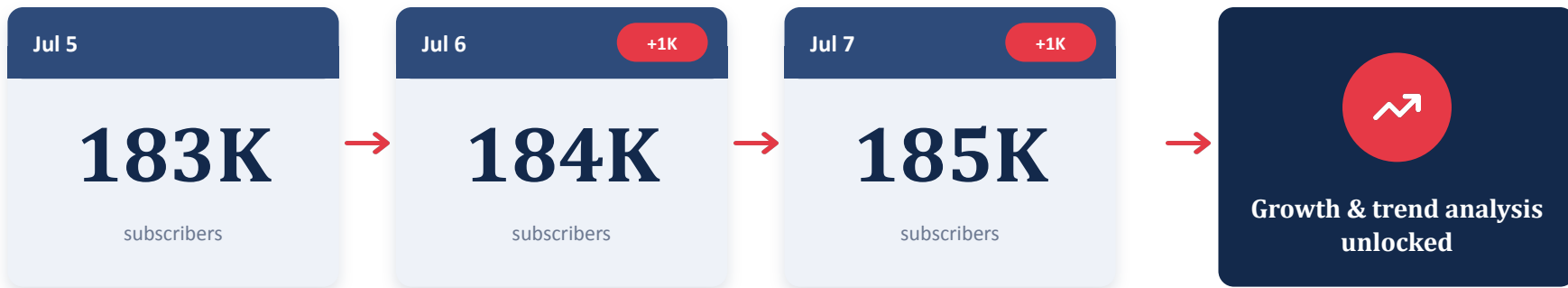
The newer 1,000-view record wins — no duplicates, and the run is safe to repeat.



Implementation: Raw + Stage → UNION ALL → ROW_NUMBER() over (video_id, snapshot_date) → CREATE OR REPLACE TABLE

Snapshot-Based History

Adding `snapshot_date` to the merge key turns daily overwrites into a time series — same day keeps the latest row, a new day adds a new one.



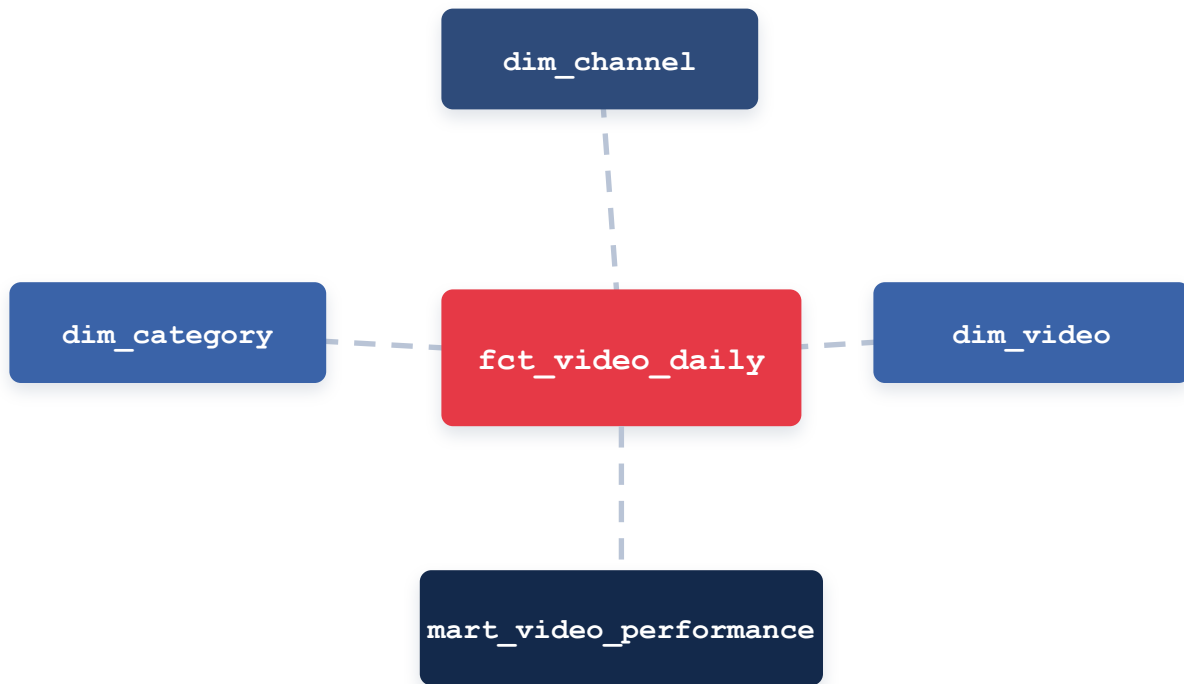
Merge key: `channel_id` + `snapshot_date` — the pipeline stays idempotent within a day while accumulating a clean daily history for growth metrics.

dbt Transformations

Why dbt? It enables modular SQL transformations, dependency management, testing, documentation and reusable models — organised into clear layers:



Star Schema



Star Schema

Central fact table surrounded by descriptive dimensions.

Simple joins, fast aggregations and BI-friendly marts sit on top.

Business Metrics Built

Beyond moving data — the pipeline computes decision-ready metrics that answer real questions about performance.



Like Rate



Comment Rate



Interaction Rate



Subscriber
Growth



Daily View
Growth



Category
Performance



Duration vs
Views



Publish-Day
Analysis

Example — $\text{Interaction Rate (\%)} = (\text{Likes} + \text{Comments}) \div \text{Views} \times 100$

Live Looker Studio Dashboards

3

Channels

719K

Subscribers

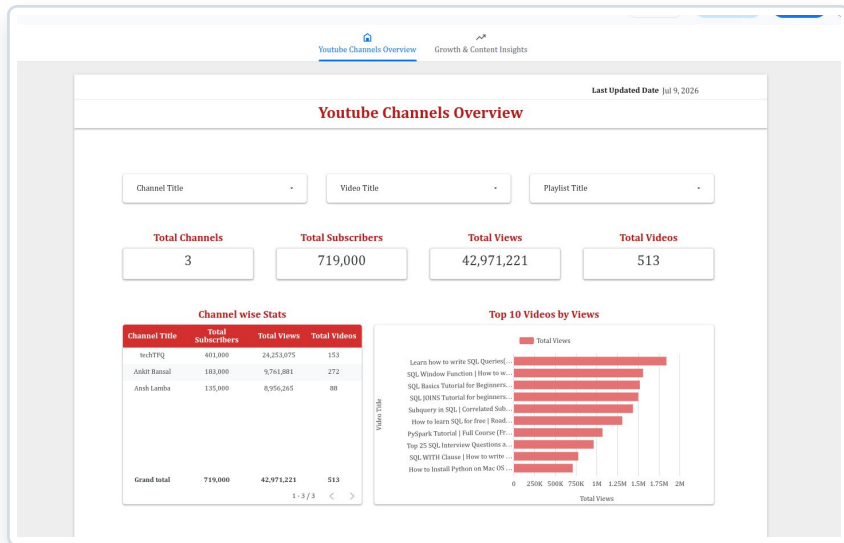
42.9M

Views

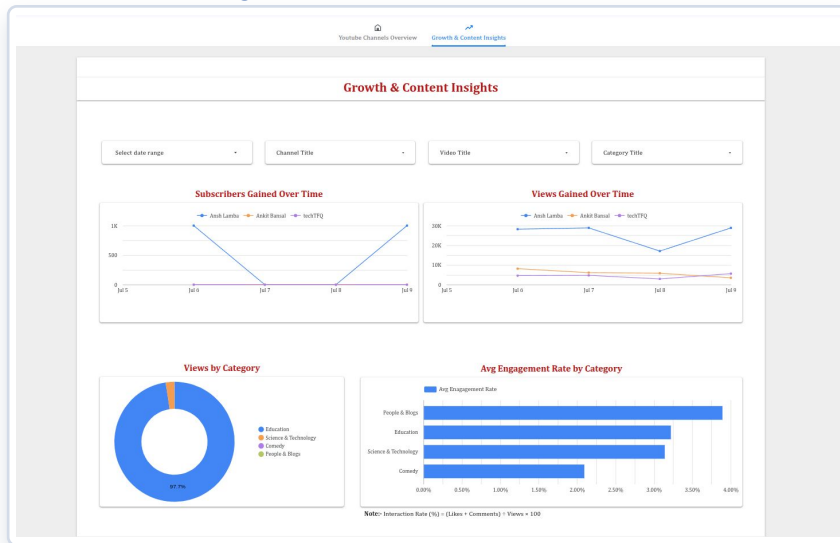
513

Videos

Youtube Channels Overview



Growth & Content Insights



► Live report: lookerstudio.google.com/reporting/c863200d-9775-41d8-878a-e63c9056a257

Challenges & Solutions



CHALLENGE

BigQuery free tier doesn't support MERGE



SOLUTION

Simulated MERGE with CREATE OR REPLACE + ROW_NUMBER() to keep the latest record.



CHALLENGE

Needed historical metrics over time



SOLUTION

Added snapshot_date to the merge key, turning overwrites into a daily time series.



CHALLENGE

Risk of duplicate rows on every rerun



SOLUTION

Built idempotent loading — reruns dedupe to one row per business key.

WHAT THIS PROJECT SHOWS

Skills Demonstrated



API integration



Modular Python



ELT architecture



BigQuery warehouse



Incremental loading



Snapshot history



Idempotent pipelines



Dimensional modeling



dbt modeling



Business metrics



BI dashboarding



Orchestration · next

Key Learnings



Scalable ELT design

Config-driven, modular ingestion that scales by editing YAML.



Reusable Python modules

One generic loader powering every entity.



Staging vs. raw discipline

Transient batches merged into persistent history.



Idempotent loading

Custom merge logic — safe to rerun, no duplicates.



Dimensional modeling

Clean dims, facts and marts via layered dbt.



Insight-driven dashboards

Looker Studio views built to answer real questions.

EXPLORE THE PROJECT

Links & Resources



[Live Dashboard](#)

lookerstudio.google.com/reporting/c863200d-9775-41d8-878a-e63c9056a257



[GitHub Repository](#)

github.com/Kausheeee/youtube-analytics-pipeline



[LinkedIn](#)

linkedin.com/in/kaushik-bendalam-650899250

Quick reference

Python

BigQuery

dbt

Looker Studio

Airflow · next

Thanks for reading — built as a portfolio analytics-engineering project.